



INSTITUTO FEDERAL DO RIO GRANDE DO NORTE – CAMPUS CAICÓ

CURSO TÉCNICO INTEGRADO EM INFORMÁTICA PARA INTERNET

Projeto de Interface de Usuário

Prof. Ciro Medeiros

10/04/2026

A6 – Exercícios de Programação com JavaScript

Instruções

1. Todas as questões devem ser respondidas em código JavaScript, salvo exceções explícitas.
2. **Não faça entrada de dados**, basta criar variáveis com valores pré-definidos para testar.
3. Essa lista serve para treinar sintaxe JavaScript, então algumas questões pedem construções específicas. Preste atenção ao enunciado.
4. Responda com cada questão em um arquivo separado (q1.js, q2.js, q3.js, ...). **Não mande em arquivo compactado.**
5. Cada questão vale 1 ponto. Total de **8 pontos**.

Questões

Q1 – Crie uma função `getStatus(condicao)` que recebe um booleano. Se `true`, retorna `Promise.resolve("Sucesso!")`. Se `false`, retorna `Promise.reject("Falha!")`. Consuma a função com `.then()` e `.catch()`.

Q2 – Crie uma função `verificarPositivo(n)` que retorna uma Promise. Dentro da Promise, implemente o seguinte comportamento:

1. Se `n` não for um `number`, rejeite com `"Tipo XX não é number."`
2. Se `n` for um `number`, mas for menor que 0, rejeite com `"0 valor XX deveria ser maior que 0"`.
3. Se `n` for um `number` positivo, apenas resolva a Promise sem devolver nada.

Na chamada da função no programa principal, exiba “Número XX é positivo.” em caso de sucesso usando o método `.then()` ou a mensagem retornada em caso de erro usando o método `.catch()`.

Q3 – Crie uma função `esperar(ms)` que recebe um tempo em milissegundos e retorna uma Promise. A Promise retornada espera o dado tempo e resolve passando o valor “Terminou!”. No programa principal, antes de chamar a função `esperar`, exiba “Esperando XXXXms...” no console. Em seguida, chame a função `esperar` e use o método `.then` para exibir a mensagem no console. O que acontece se você exibir algo depois da chamada à função `esperar`? Experimente.

Q4 – Crie uma Promise que resolve com o número 5. Encadeie dois `.then()`:

- Primeiro: recebe o número e retorna seu dobro.
- Segundo: recebe o resultado anterior e retorna seu triplo.

Exiba o resultado final (deve ser 30).

Q5 – Reutilize a função da questão Q1. Crie uma lista com 3 Promises de `getStatus` passando `true` em todas e execute com `Promise.all`. O que acontece se você passar `false` para uma delas? Experimente.

Q6 – Reutilize o código da questão Q5, porém agora execute a lista de Promises com `Promise.allSettled`. O que acontece agora se passar `false` para uma delas? Experimente.

Q7 – Adapte o código da questão Q6 de maneira que a função `getStatus` receba um `id` e um booleano `condicao` e retorne uma Promise com o seguinte comportamento:

1. Esperar um tempo aleatório entre 1 e 3 segundos.
2. Resolver com “(id) Sucesso :)” ou rejeitar com “(id) Erro :(” de acordo com o booleano informado.

Instancie 3 Promises (ids 1, 2 e 3) e execute com `Promise.any`. Experimente colocar todos os valores booleanos `true`, todos `false` e misturá-los. Analise o resultado.

Q8 – Repita o código da questão Q7, porém execute com `Promise.race`. Analise o resultado.